

Class 25: Requirements Engineering in Practice

Session 25 of 36 – Hands-on Workshop

Data-driven Systems Engineering

Agenda

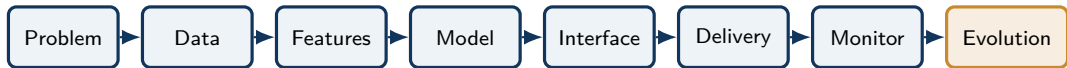
- Recap: what makes a requirement good
- Workshop structure and case assignment
- Elicitation: asking the right questions
- Writing functional and non-functional requirements
- Surfacing conflicts and trade-offs
- Peer review against a validation checklist

From theory to a written specification

Learning goals

- Apply elicitation techniques to a concrete case in one working session.
- Draft testable functional and non-functional requirements from a vague prompt.
- Identify a genuine conflict between requirements and propose an explicit trade-off.
- Use a validation checklist to critique a peer team's requirements.

Course Map and Running Cases



Today in the course

Focus: Turning requirements theory into a written, testable specification.

Bridge: This session applies Class 23's theory, sharpened by YesAlps' real-world session in Class 24, to a hands-on writing exercise instead of another lecture.

Fraud case

Fraud track: write functional and non-functional requirements for the fraud scoring service, from ingestion to analyst review.

Pasteurization case

Pasteurization track: write functional and non-functional requirements for the abnormal-state alerting dashboard.

Course thread: Requirements written today should be traceable to the tests, deployment gates, and monitoring rules the course covers later.

Recap: qualities of a good requirement

- Necessary, testable, unambiguous, feasible, traceable.
- A requirement is not a feature wish; it is something a reviewer could check.

Weak versus better, one more time

“The system should be reliable” versus “the stream endpoint shall recover and resume within 5 seconds after a dropped connection.”

Workshop structure

- Form small groups; each group picks one track: fraud scoring or pasteurization alerting.
 - Time-box each stage; a rough spec beats an unfinished perfect one.
 - Every stage below produces a short written artifact the group keeps.
1. Elicit (10 min)
 2. Draft functional requirements (10 min)
 3. Draft non-functional requirements (10 min)
 4. Surface one conflict (5 min)
 5. Peer review with the checklist (10 min)

Elicitation is structured discovery

- Elicitation means discovering requirements from users, operators, policies, data constraints, and business goals.
- Interviews, observation, document review, and prototype walkthroughs are all useful methods.
- The purpose is not only to collect wishes, but to reveal conflicts and hidden assumptions.

Good elicitation prompts for project teams

- Who acts on the prediction, and what action do they take?
- What happens if the model is unavailable for one hour?
- Which inputs may arrive late, missing, or malformed?
- When do labels appear, and who trusts them?
- Which requirement is actually a policy or compliance constraint?

Exercise 1: Elicitation notes

- Answer the prompts on the previous slide for your chosen track.
- Write raw notes, not polished sentences; contradictions between teammates are useful signal, not noise.
- Flag anything that sounds like a policy or domain constraint rather than a feature request.

From notes to functional requirements

Raw note

“The analyst needs to know why a transaction was flagged.”

Functional requirement

The API shall return, alongside the score, the top 3 contributing features for each flagged transaction.

- A functional requirement names an observable system behavior, not a feeling.

Exercise 2: Write three functional requirements

- Using your elicitation notes, write three functional requirements for your track.
- Each one should name an input, an action, and an observable output.
- Avoid the words “should” and “easy”; prefer “shall” plus a concrete behavior.

Non-functional requirements deserve equal weight

Category	Question to answer with a number
Latency	How fast must a response or alert be, and for what fraction of cases?
Availability	How much downtime is tolerable, and what happens during it?
Auditability	What must be logged, and for how long?
Privacy	What data can be stored, shown, or shared, and with whom?

Exercise 3: Write two non-functional requirements

- Pick two categories from the table and write one requirement each.
- Every requirement must include a number: a time, a percentage, a retention period, or a rate.
- If you cannot put a number on it yet, write down what you would need to know to add one.

Conflicts between requirements are normal

- Fast response may conflict with more detailed explanations.
- High recall may conflict with low false positive rates.
- Privacy constraints may limit what data can be stored for monitoring or audit.
- Requirements engineering helps teams make these trade-offs explicit.

Exercise 4: Name one conflict

- Look at your functional and non-functional requirements together.
- Find one real tension between them, for example explanation detail versus latency.
- Write one sentence stating which side you would favor by default, and why.

Validation techniques that fit student projects

- Requirement reviews with a checklist for ambiguity and measurability.
- Use-case walkthroughs for edge cases and operator behavior.
- Prototype validation with a minimal dashboard or API client.
- Acceptance test drafting before full implementation.

Exercise 5: Peer review

- Swap your written requirements with another group.
- For each requirement, check: necessary, testable, unambiguous, feasible, traceable.
- Mark anything that fails a check and suggest a one-line fix.

Ground your requirement in something real

Repo activity

- Fraud track: anchor requirements to the fields the API already accepts and returns, as seen in 'Class6_model_api.py'.
- Pasteurization track: anchor requirements to the fields already streamed by 'pasteurization/serving.py', such as state, T, pH, and dTdt.
- A requirement that cannot be tied to an existing field or endpoint is either a new feature request or still too vague.

Question

Which of your non-functional requirements would be hardest to test automatically, and why?

- This surfaces the gap between writing a requirement and actually verifying it.
- It is a preview of the testing and monitoring gates later in the course.

What a strong requirements artifact looks like

- A short list of functional requirements naming input, action, and output.
- A short list of non-functional requirements, each with a number attached.
- At least one documented conflict and the team's default resolution.
- Peer-review marks showing the requirements survived a second reading.

Papers and materials

Sommerville, Software Engineering; IEEE recommended practice for software requirements specifications; User stories and acceptance criteria guide; Practical overview of functional and non-functional requirements.

From This Class to the Next

What stays with us

- A requirement earns its place only if someone could check whether it was met.
- Elicitation, drafting, and peer review are fast enough to run in a single session.
- Conflicts between requirements are normal and should be written down, not hidden.

Project use

Keep today's written requirements; they are the seed of the project proposal work in Class 32 and Class 34.

Next connection

Class 26 moves from what the system must do to how the team organizes work to build it, starting with process models and agile foundations.

Course thread: Today's artifact is the first concrete deliverable in the course thread that eventually becomes each team's own project.